SOFTWARE LAB – 9

TEAM -

ARYAN RAJ – 24BDS1094

YASH RAJ – 24BDS1209

AIM – DRAW THE CLASS DIAGRAM FOR SOFTWARE PROJECT

1. Description of Classes

1. User

Represents all users of the YieldnYarn system, including sellers, buyers, and staff. It stores user details, contact information, and role information. Users can create listings, place bids, and participate in orders.

2. Listing

Represents recyclable material listed by a seller for sale. It stores details such as material type, quantity, location, asking price, and listing status. Each listing belongs to a seller and can have multiple photos and a grading result.

3. ListingPhoto

Stores photographs uploaded for a material listing. These photos are used by the AI grading system to assess the material's quality. Each photo is associated with one listing.

4. GradingResult

Stores the results of AI-based material quality assessment, including quality scores, grade details, confidence information, and verification status. Staff can verify or correct the results when manual review is required.

5. MarketPrice

Stores market price information for different types and grades of recyclable materials. This information helps sellers determine reasonable asking prices and supports price estimation.

6. BuyerRequirement

Represents the material requirements submitted by buyers. It stores the required material type, quantity, acceptable quality, and target price. These requirements help match buyers with suitable seller listings.

7. Bid

Represents an offer or counter-offer between a buyer and a seller for a material listing. It stores the offered price, quantity, and bid status. Bids can lead to an order when the parties agree.

8. Order

Represents a confirmed transaction between a buyer and a seller or multiple sellers. It store order details, total quantity, total amount, and order status.

9. OrderAllocation

Represents a portion of an order assigned to a particular seller and listing. It records the allocated quantity and associated amount. This supports orders fulfilled using material from multiple sellers.

10. Complaint

Stores complaints raised by users regarding listings, grading, bids, or orders. It records the complaint description, status, and resolution details to support issue tracking and resolution.

2. Relationships Between Classes

1. User — Listing: One-to-Many (1:N) — One seller can create multiple listings; each listing belongs to one seller.

2. Listing — ListingPhoto: One-to-Many (1:N) — One listing can contain multiple photos; each photo belongs to one listing.

3. Listing — GradingResult: One-to-One (1:1) — A listing can have one grading result, which stores its quality assessment.

4. MarketPrice — Listing: One-to-Many (1:N) — One market price record can be used to compare or estimate prices for multiple listings of the corresponding material and grade.

5. User — BuyerRequirement: One-to-Many (1:N) — One buyer can create multiple requirements; each requirement belongs to one buyer.

6. Listing — Bid: One-to-Many (1:N) — One listing can receive multiple bids; each bid is associated with one listing.

7. User — Bid: One-to-Many (1:N) — A user can participate in multiple bids as a buyer or seller; each bid records the participating users.

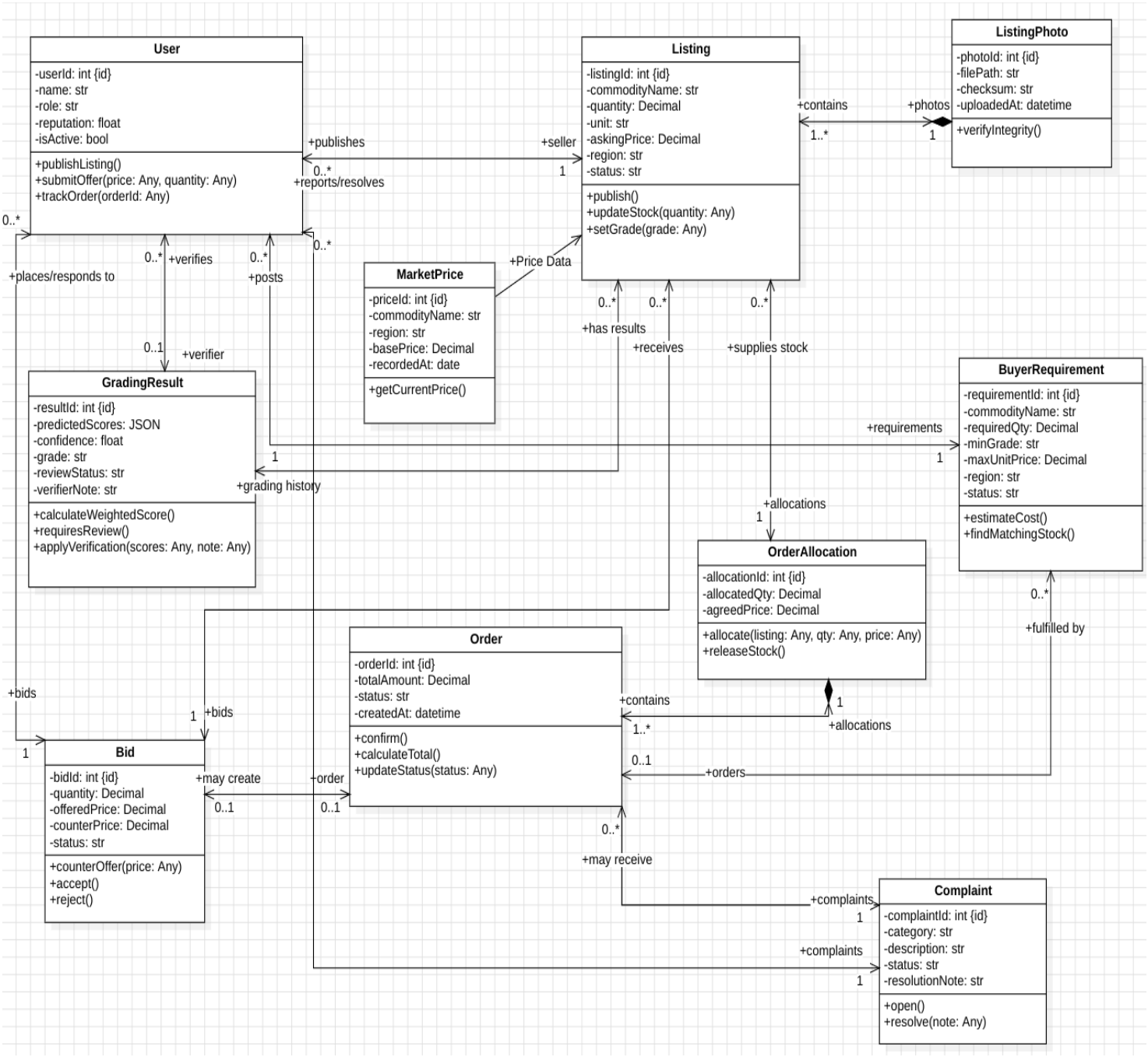
8. Bid — Order: One-to-One (1:1) — An accepted bid can result in an order.

9. Order — OrderAllocation: One-to-Many (1:N) — One order can have multiple allocations; each allocation belongs to one order.

10. User — Complaint: One-to-Many (1:N) — One user can raise multiple complaints; each complaint is raised by one user.

11. Order — Complaint: One-to-Many (1:N) — One order can have multiple complaints; each complaint can be associated with an order.

3. CLASS DAIGRAM (NEXT PAGE)


User
-userId: int {id}
-name: str
-role: str
-reputation: float
-isActive: bool
+publishListing()
+submitOffer(price: Any, quantity: Any)
+trackOrder(orderId: Any)
Listing
-listingId: int {id}
-commodityName: str
-quantity: Decimal
-unit: str
-askingPrice: Decimal
-region: str
-status: str
+publish()
+updateStock(quantity: Any)
+setGrade(grade: Any)
ListingPhoto
-photoId: int {id}
-filePath: str
-checksum: str
-uploadedAt: datetime
+verifyIntegrity()
+contains
+photos
1..*
1
+publishes
+seller
0..*
1
+reports/resolves
0..*
+places/responds to
0..*
+verifies
0..1
+verifier
0..*
+posts
0..*
MarketPrice
-priceId: int {id}
-commodityName: str
-region: str
-basePrice: Decimal
-recordedAt: date
+getCurrentPrice()
+Price Data
0..*
+has results
0..*
+receives
0..*
+supplies stock
GradingResult
-resultId: int {id}
-predictedScores: JSON
-confidence: float
-grade: str
-reviewStatus: str
-verifierNote: str
+calculateWeightedScore()
+requiresReview()
+applyVerification(scores: Any, note: Any)
1
+grading history
BuyerRequirement
-requirementId: int {id}
-commodityName: str
-requiredQty: Decimal
-minGrade: str
-maxUnitPrice: Decimal
-region: str
-status: str
+estimateCost()
+findMatchingStock()
+requirements
1
+allocations
1
OrderAllocation
-allocationId: int {id}
-allocatedQty: Decimal
-agreedPrice: Decimal
+allocate(listing: Any, qty: Any, price: Any)
+releaseStock()
0..*
+fulfilled by
Order
-orderId: int {id}
-totalAmount: Decimal
-status: str
-createdAt: datetime
+confirm()
+calculateTotal()
+updateStatus(status: Any)
+contains
1..*
1
+allocations
0..1
+orders
+bids
1
+bids
1
Bid
-bidId: int {id}
-quantity: Decimal
-offeredPrice: Decimal
-counterPrice: Decimal
-status: str
+counterOffer(price: Any)
+accept()
+reject()
+may create
0..1
+order
0..1
0..*
+may receive
+complaints
1
+complaints
1
Complaint
-complaintId: int {id}
-category: str
-description: str
-status: str
-resolutionNote: str
+open()
+resolve(note: Any)